

This Lab's Objectives

1. Understand a Netkit lab
 2. Listening and logging Internet communications between virtual machines.
 3. Analyse the HTTP Request/Response cycle.
-

Expected Outcomes

After the laboratory session, students should be able to:

- launch a group of Netkit virtual machines configured as a Netkit lab using *lstart*.
 - execute a simple client/web-server Netkit lab
 - use tools to capture the HTTP conversation
 - analyse and be able to explain each step of the HTTP Request and Response cycle
-

Workplan

These lab worksheets assume that you will make accurate notes of everything that you do for later consideration and revision. You should use your lab diary to note down items such as commands used, the outcome of each command, and answers to questions posed in these instructions.

All lab machines have Cherry Notes software installed, which facilitates organised note-taking. Alternatively, you could buy a hardback A4 laboratory notebook.

NB this is a complex lab – please read each instruction carefully – if you miss out any steps, then don't expect to achieve the correct results! You will need to have completed and thoroughly understood all the steps before the next lab session.

Remember that there is a lab work wiki available to you on BB. Please use this to share and communicate with you colleagues.

Make sure you have completed all the steps in this worksheet before the next lab session.

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

Starting assumptions

1. You have completed all the work listed in the previous lab sessions
 - You are familiar with basic Linux commands used to inspect and manipulate directories and files.
 - You have used Netkit *lstart* to launch a coordinated group of several virtual machines. Specifically, you are aware of how this group is coordinated via the *lab.conf* file.
 - You know how to shut down virtual machines using *lhalt* and *lcrash*.
 - You have used *ifconfig* and *ping* within the virtual machines.
 - You have saved captured network traffic from a virtual machine using *tcpdump*.
 - You have viewed a packet capture file in the real Linux host using *wireshark*.

Activities

Reminder

- You need to make accurate notes of what you do.
- You will complete unfinished activities before the next timetabled lab session.
- You will resolve what you do not understand by conducting your own careful (and ethically sound) experimentation and / or further reading.

Netkit Labs

The example labs from lab 11 onwards are manipulated using the Netkit *l.....* commands, such as [lstart](#). Multiple xterms are started simultaneously and closed, all with a single commend for each process.

A directory containing a complete Netkit lab will typically contain:

- one or more directories (usually empty)
 - one for each virtual machine
- one or more *.startup* files
 - again, one for each xterm
- one *lab.conf* file
 - configures the ethernet connections and subnets for all machines.
- one *lab.dep* file
 - speeds things up by starting the machines in parallel.

Getting Started

2. If your computer is switched off, now turn it on.
 - If necessary, turn off your PC and reboot into Linux.

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

3. Create a directory for this lab's work (*ctec1704/lab_11*) and move into it.

```
cd ~
mkdir -p ctec1704/lab_11
cd ctec1704/lab_11
```

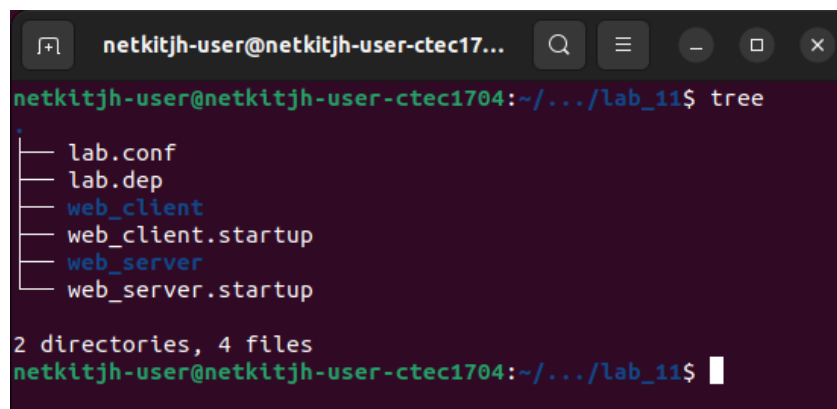
4. Download the zipped archive *netkit_lab_11.tar.xz* from Blackboard.
- This contains the configuration information for just 2 xterms – *web-server* and *web-client*.
5. Extract the contents of the archive file, and make sure that all the files and directories for the lab are in the new directory (you may need to move the lab files to the correct directory). NB Make sure you extract the files in the terminal. Sometimes using the files window causes problems ...

```
.../lab_11$ tar -xvf netkit_lab_11.tar.xz
```

6. List the contents of the directory with the *ls* command. You will see files and directories to create 2 xterms.

```
.../lab_11$ ls -l
```

7. You will see
- one lab.conf file
 - one lab.dep file
 - two directories (both empty)
 - two *.startup files
8. The Linux command “tree” can be used to display the contents of the directory:



```
netkitjh-user@netkitjh-user-ctec1704:~/.../lab_11$ tree
.
├── lab.conf
├── lab.dep
├── web_client
├── web_client.startup
├── web_server
└── web_server.startup

2 directories, 4 files
netkitjh-user@netkitjh-user-ctec1704:~/.../lab_11$
```

lab.conf

Use the *cat* command to review the contents of a file. NB only ever open a file in a text editor if you mean to make some changes to it.

```
cat lab.conf
```

This file contains instructions to pre-configure the network interface settings for each virtual machine. The *man lab.conf* command will give you further details as to how this file works.

9. When you see a line in *lab.conf* such as:

```
web-client[0]=lan
```

it is equivalent to the terminal command:

```
vstart web-client -eth0=lan
```

- *web-client* is the name of a network machine (in an xterm)
- the digits in the square brackets represent a network interface card
- *lan* represents a collision domain (in effect a network cable to allow connections between network cards on multiple network machines)

*.startup files

10. There is one startup file for each Netkit virtual machine.

These files contain the final instructions that are executed as each xterm starts, before you are presented with the login prompt of the virtual machine. The startup files for the web-client and web-server are very similar - they are only two lines long.

Look at the content of each of these files in turn and draw a simple network diagram in your notes. Ensure you get the network card designation (eth0) and the IP address recorded accurately. Check your diagram with your lab tutor.

Analysing the HTTP Request/Response Cycle

11. With your current directory as the *lab_11* directory (which contains the *lab.conf* and *lab.dep* files), launch the two virtual machines using the instruction:

```
.../lab_11$ lstart
```

12. You should see both of the virtual machines start up in quick succession. Once they have started, move them to positions on the screen that match the diagram.

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

13. Use the *ping* command to check that the two xterms (*web-client* and *web-server*) are able to exchange messages.
- Hint: the IP address for each xterm is in the relevant *startup* file.

```
root@web-client:~# ping -c 2 <<IP>>
```

Starting the Apache web server

The Apache 2 web server is pre-installed in all Netkit hosts.

14. The web server is not yet running – use the status command below to show this. In the *web-server* xterm window, enter:

```
root@web-server:~# systemctl status apache2
```

Output:

```
web-server
root@web-server:~# systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; disabled; vendor preset: enabled)
   Active: inactive (dead)
     Docs: https://httpd.apache.org/docs/2.4/
root@web-server:~#
```

15. Start the web server daemon with the following:

```
root@web-server:~# systemctl start apache2
```

16. Rerun the status check as above to check that the web server process is now running.
- What is a *pid* in this context?

```
web-server
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; disabled; vendor preset: enabled)
   Active: active (running) since Fri 2023-05-05 15:45:03 UTC; 26s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 1612 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
 Main PID: 1623 (apache2)
    Tasks: 55 (limit: 142)
   Memory: 7.4M
      CPU: 1.290s
   CGroup: /system.slice/apache2.service
           └─1623 /usr/sbin/apache2 -k start
             └─1624 /usr/sbin/apache2 -k start
               └─1625 /usr/sbin/apache2 -k start

May 05 15:45:03 web-server systemd[1]: Starting The Apache HTTP Server...
May 05 15:45:03 web-server apachectl[1622]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name,
May 05 15:45:03 web-server systemd[1]: Started The Apache HTTP Server.
```

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

17. Locate and inspect the contents (how?) of the default web page – *index.html* - in the file system on the *web-server* xterm (*/var/www/html*). This directory is known as the *web root* of a web-site.

Accessing a web page with Telnet

The Telnet command can be used to check that the web server is behaving correctly by requesting the default web page. In the *web-client* xterm window:

```
root@web-client:~# telnet 192.168.150.2 80
```

This tells the web-client machine to use the Telnet application to access the *web server daemon* (how/why?) on port 80.

18. Assuming that the web server is running, you will get a reply:

```
Trying 192.168.150.2...
Connected to 192.168.150.2.
Escape character is '^['.
```

19. Press *Enter*. This gives blank line on the terminal – this is a requirement of the HTTP protocol.
20. Enter the following:

```
GET / HTTP/1.0
```

21. Press *Enter* a couple of times, and you will see the HTTP response message and the HTML web page code contained in the *index.html* file. The HTML code section is the *payload*.

```
HTTP/1.1 200 OK
Date: Fri, 16 Sep 2016 15:35:32 GMT
Server: Apache/2.2.9 (Debian)
Last-Modified: Sat, 06 Dec 2007 16:48:06 GMT
ETag: "4006-2d-45d638ed03d80"
Accept-Ranges: bytes
Content-Length: 45
Vary: Accept-Encoding
Connection: close
Content-Type: text/html
```

```
<DOCTYPE html ... etc
(multiple lines of HTML code)
```

```
</html>
```

```
Connection closed by foreign host.
```

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

22. Analyse the separate items of the GET request, and each line of the response.
23. The Apache service can be restarted if required (eg after configuration changes):

```
root@web-server:~# systemctl restart apache2
```

24. To stop the web server:

```
root@web-server:~# systemctl stop apache2
```

25. To see all of the commands that *systemctl* provides:

```
root@web-server:~# systemctl --help
```

Making Web Server Configuration Changes

The warning message, seen when starting the Apache web server, “*Could not reliably determine the servers name...*” occurs because the the name of the server has not been added to the Apache configuration. Therefore this message will be displayed each time the web server is started and/or stopped. The *httpd.conf* file can be updated to set a value to remove the message.

26. Close the lab.

```
~/.../lab_11$ lcrash  
~/.../lab_11$ lclean
```

27. You need to create the *httpd.conf* file in your **host** system so that it is copied into the *web-server* xterm when it is executed.

```
~/.../lab_11$ mkdir -p web-server/etc/apache2/  
~/.../lab_11$ gedit web-server/etc/apache2/httpd.conf &
```

NB the ‘&’ character opens the editor as a ‘background’ process. This means that the host terminal prompt is still available to enter other commands whilst the file is being edited.

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

28. Enter the following text into the *httpd.conf* file.

```
ServerName 127.0.0.1
```

Save and close the file when finished.

29. Restart the lab and then start Apache to check the warning message has gone.

It is also possible to start Apache automatically when the *web-server* xterm starts.

30. Close the lab as before.

31. Edit the file *web-server.startup* and add the following below the *ip* command:

```
systemctl start apache2
```

32. Restart the lab and check the status of the web server.

```
root@web-server:~# systemctl status apache2
```

33. Use the Telnet command again in the web-client to check that the client is still able to download the *index.html* file.

Other HTTP methods

Earlier you used the GET method of HTTP to access the remote web server. There are several other methods that HTTP recognises:

34. Use the *telnet* command to test the POST, HEAD and TRACE methods of the HTTP protocol. In the *web-client* xterm, enter:

```
web-client:~# telnet 192.168.150.2 80
Trying 192.168.150.2...
Connected to 192.168.150.2.
Escape character is '^]'.

POST / HTTP/1.0
```

You should see the following response:

```
HTTP/1.1 200 OK
Date: Fri, 16 Sep 2016 15:57:29 GMT
Server: Apache/2.2.9 (Debian)
Last-Modified: Sat, 06 Dec 2007 16:48:06 GMT
ETag: "4006-2d-45d638ed03d80"
Accept-Ranges: bytes
Content-Length: 45
Vary: Accept-Encoding
Connection: close
Content-Type: text/html

<DOCTYPE html ... etc
(multiple lines of HTML code)

</html>

Connection closed by foreign host.
web-client:~#
```

35. Repeat with the HEAD and TRACE methods.
36. Compare the output from each method with the output from the GET method above.
37. Repeat the Telnet command but with deliberate (forced) errors (change case, etc). How does the output change in each case?

NB The GET and POST methods are the only methods that browsers, such as Firefox, will ever use.

Capture an HTTP request/response cycle

You are going to capture and analyse a complete web page request and response cycle of messages.

38. Restart tcpdump on the web-server xterm to verbosely capture packets and write them to a file in your *captures* directory on the host system. Use a suitable and relevant *pcap* filename.

```
root@web-server:~# tcpdump -v -i eth0 -s0 -w
/hosthome/ctec1704/captures/http1.pcap
```

Using the lynx text-based browser

There is a text-based web browser called *lynx* available in the Netkit xterms. This can be run in the *web-client* xterm to interact with the Apache web server process running in the *web-server* xterm.

39. Question: Lynx does not format web-pages with fonts or colour, or displays images, so who would need a text-based browser?
40. Start the *lynx* browser in the *web-client* xterm terminal:

```
root@web-client:~# lynx 192.168.150.2
```

41. You will see the response below in the web-client xterm.

```

Apache2 Debian Default Page: It works (pl of 3)
Debian Logo Apache2 Debian Default Page
It works!

This is the default welcome page used to test the correct operation of the
Apache2 server after installation on Debian systems. If you can read this
page, it means that the Apache HTTP server installed at this site is working
properly. You should replace this file (located at /var/www/html/index.html)
before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is
about, this probably means that the site is currently unavailable due to
maintenance. If the problem persists, please contact the site's
administrator.
Configuration Overview

Debian's Apache2 default configuration is different from the upstream default
configuration, and split into several files optimized for interaction with
Debian tools. The configuration system is fully documented in
/usr/share/doc/apache2/README.Debian.gz. Refer to this for the full
documentation. Documentation for the web server itself can be found by
accessing the manual if the apache2-doc package was installed on this server.

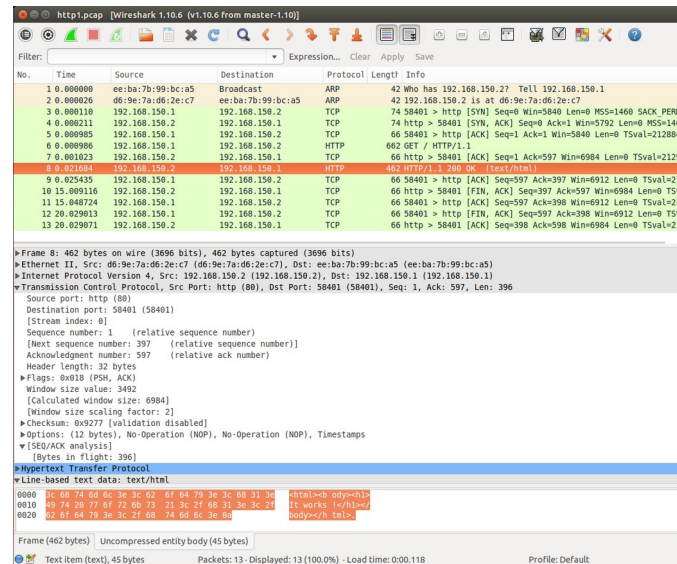
The configuration layout for an Apache2 web server installation on Debian
systems is as follows:
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf
--

-- press space for next page --
Arrow keys: Up and Down to move. Right to follow a link; Left to go back.
```

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

42. Exit from *lynx* (how?)
43. Stop the tcpdump capture.
44. Open the pcap file in Wireshark as in the previous lab.
45. Analyse the captured packet stream with Wireshark (there will be about 14 packets). You will be able to see the complete TCP conversation.



- Locate the packets that carry the:
 - HTTP request
 - HTTP response with payload
 - The TCP header contains information that enables the communication between the two devices. To view it, look in the lower window and locate the heading Transmission Control Protocol. Select that, and the header field will be displayed.
 - Find the TCP header field in each packet containing the status flags in the SYN, SYN/ACK and FIN packets.
 - Note the settings of the status bits.

Updating the index.html file

The pre-written index.html is very long. It can be simplified:

46. Recreate the *index.html* file for the web-server on the **host** system, and open it for editing.

```
~/.../lab_11$ mkdir -p web-server/var/www/html
~/.../lab_11$ gedit web-server/var/www/html/index.html &
```

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

47. Enter the code for the simple web page

```
<html>
  <body>
    <h1>It works!</h1>
  </body>
</html>
```

48. Add a paragraph of text below the header tags – use <p></p>

```
<html>
  <body>
    <h1>It works!</h1>
    <p>Hello there!</p>
  </body>
</html>
```

49. Restart the lab (the '&&' allows two netkit commands to be “daisy chained” together.)

```
~/.../lab_11$ lcrash && lstart
```

50. Check that your edited file is now available by using *lynx*.

Multiple web pages

51. Create a second web page in the same directory on your host system as *index.html*

```
~/.../lab_11$ gedit web-server/var/www/html/page2.html &
```

52. Use the text below as the content of the new web page.

```
<html>
  <body>
    <h1>This is page 2</h1>
    <a href="index.html">Return to the home page</a>
  </body>
</html>
```

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

53. Add an HTML page link tag in *index.html* to link to the newly created web page.

```
<html>
  <body>
    <h1>It works!</h1>
    <p>Hello there!</p>
    <a href="page2.html">Page 2</a>
  </body>
</html>
```

54. Restart the lab, then capture the HTTP packets on the web server as usual. Make sure that you click on the HTML link to access the second file that you created.
55. Analyse the result in Wireshark, as before. Can you see the complete cycle for both pages in your capture?

Using nmap

[nmap](#) is another really useful network tool; this tool scans an IP address, or a range of IP addresses, to detect open ports and the services available on a server, eg web server, database server, etc. The home web-site for nmap is [here](#).

56. Start another capture in the web-server – *nmap1.pcap*.
57. Run *nmap* in the web-client xterm to scan the web server:
- what effect do the switches have?

```
root@web-client:~# nmap -v -sV 192.168.150.2
```

58. Be patient – the scan may take some time.
59. When the scan is complete, read through the output (see below) in the web-client xterm.
60. As before, analyse the capture in Wireshark, particularly any *status bit* settings.
- NB the last few packets captured show how the HTTP service is scanned – look for the 3-way handshake packets.
61. Use the relevant *man* page to deconstruct the switches for the *nmap* command. Are there any other useful switches that would capture more information for analysis?

Lab Worksheet 11

The Application Layer - HTTP & Apache - the HTTP Request/Response Cycle

```

web_client:~$ nmap -v -sV 192.168.150.2

Starting Nmap 4.68 ( http://nmap.org ) at 2021-02-23 10:07 UTC
Initiating ARP Ping Scan at 10:07
Scanning 192.168.150.2 [1 port]
Completed ARP Ping Scan at 10:07, 0.04s elapsed (1 total hosts)
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled.
Try using --system-dns or specify valid servers with --dns-servers
Initiating SYN Stealth Scan at 10:07
Scanning 192.168.150.2 [1715 ports]
Discovered open port 80/tcp on 192.168.150.2
Discovered open port 111/tcp on 192.168.150.2
Completed SYN Stealth Scan at 10:07, 1.30s elapsed (1715 total ports)
Initiating Service scan at 10:07
Scanning 2 services on 192.168.150.2
Completed Service scan at 10:07, 6.01s elapsed (2 services on 1 host)
SCRIPT ENGINE: Initiating script scanning.
SCRIPT ENGINE: error while initializing script rules:
/usr/share/nmap/scripts/skype_v2-version.nse:6: module 'comm' not found:
  no field package.preload['comm']
  no file '/usr/share/nmap/nselib/comm.lua'
  no file './comm.lua'
  no file '/usr/local/share/lua/5.1/comm.lua'
  no file '/usr/local/share/lua/5.1/comm/init.lua'
  no file '/usr/local/lib/lua/5.1/comm.lua'
  no file '/usr/local/lib/lua/5.1/comm/init.lua'
  no file '/usr/lib/nmap/nselib-bin/comm.so'
  no file './comm.so'
  no file '/usr/local/lib/lua/5.1/comm.so'
  no file '/usr/local/lib/lua/5.1/loadall.so'
stack traceback:
  [C]: in function 'require'
  /usr/share/nmap/scripts/skype_v2-version.nse:6: in main chunk
  [C]: ?
  [C]: in function 'Entry'
  /usr/share/nmap/scripts/script.db:11: in main chunk
  [C]: ?
  [C]: ?

SCRIPT ENGINE: Aborting script scan.
Host 192.168.150.2 appears to be up ... good.
Interesting ports on 192.168.150.2:
Not shown: 4713 closed ports
PORT      STATE SERVICE VERSION
80/tcp    open  http   Apache httpd 2.2.9 ((Debian))
111/tcp   open  rpcbind
MAC Address: FA:5D:6A:88:71:93 (Unknown)

Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at http://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.720 seconds
Raw packets sent: 1723 (75.810KB) | Rcvd: 1716 (68.650KB)
web_client:~$

```

62. Now review what you have learned in this lab session.
63. Remember to *lcrash* the lab when you have finished. Check that the .disk files have been removed.

Follow-up work

Look up the HTTP header fields – explain the need for each, and be able to give example values. Therefore, explain how a web browser requests a web server to serve (send) web pages.

You could start by reading the excellent summary at <https://en.wikipedia.org/wiki/Http>. However, you should access the reference links within the page for more details.

You should also access RFC 2616 for HTTP (<https://tools.ietf.org/html/rfc2616>). RFCs can be heavy reading, but as an IT professional, you need to acquire the ability to select the relevant passages.

Make sure you have completed all the steps in this worksheet before the next lab session.